Copiloto de movilidad del Valle de Aburrá — Informe final

Curso: SI3003 – Inteligencia Artificial (EAFIT, 2026-1) · Proyecto final integrador **Entrega:** 13 de noviembre de 2026

> Informe breve (máx. 4 páginas). Para la demostración y las instrucciones de ejecución, ver `README.md`. Para la declaración de uso de IA generativa, ver `DECLARACION_IA.md`.

---

1. Problema

En el Valle de Aburrá un viajero elige entre metro, bus, bicicleta y carro según objetivos que compiten: **tiempo**, **costo** y **emisiones de CO_2**. Además hay restricciones del contexto (horario del Metro, pico y placa, hora pico). El objetivo del proyecto es un **copiloto de movilidad** que (a) calcule rutas óptimas multimodales y multiobjetivo sobre un grafo del Valle de Aburrá, y (b) ofrezca un **agente conversacional** que responda preguntas de movilidad apoyándose en herramientas y en un corpus documental (RAG), citando sus fuentes.

Alcance del MVP: la red base es la **Línea A del Metro** (21 estaciones, Niquía–La Estrella) más corredores reales de bus, bicicleta y carro entre estaciones. El grafo es una simplificación: los nodos son estaciones y las aristas representan tramos entre ellas.

2. Datos

- **Estaciones y geometría:** las 21 estaciones de la Línea A con coordenadas aproximadas (`core/graph.py`, `STATIONS`). Verificables contra OSM/osmnx.
- **Rutas de bus (`data/bus_routes.json`):** tres corredores reales del Valle de Aburrá (La 50 al sur; Envigado–Poblado–centro; Autopista Norte), con operador, tarifa 2026 y fuente. Las paradas se mapean a las estaciones de la Línea A que cada corredor sirve.
- **Tarifas (reales, 2026 – SITVA):** Metro perfil Frecuente con Cívica **$3.820** y bus colectivo metropolitano **$3.800**, vigentes desde el 10-ene-2026 (Área Metropolitana del Valle de Aburrá). Fuentes y perfiles adicionales (estudiantil, adulto mayor, eventual, discapacidad) en `corpus/metro_medellin.md`.
- **Horario del Metro:** L-S 4:30–23:00; dom/festivos 5:00–22:00 (`METRO_HOURS`), con fuente en el corpus.
- **Pico y placa (2.º semestre 2026):** rotación oficial por dígito, horario, vías y vehículos exentos y sanción (`corpus/pico_y_placa.md`), con fuentes de la Secretaría de Movilidad de Medellín / Área Metropolitana.
- **Emisiones y velocidades: supuestos del modelo** documentados en `corpus/emisiones_y_supuestos.md` (metro 30, bus 80, bici 0, carro 170 gCO_2/km; velocidades de flujo libre y reducción en hora pico).

3. Método — Parte 1 (búsqueda)

Se implementan **desde cero** (`core/search.py`) tres algoritmos sobre el grafo multimodal:

- **A*** multiobjetivo con una heurística admisible y consistente: tiempo mínimo en línea recta a la velocidad máxima disponible (`w_time · haversine/v_max`). Nunca sobreestima el costo real, por lo que A* es óptimo.
- **Dijkstra / UCS**: equivalente a A* con heurística 0 (`use_h=False`).
- **BFS**: minimiza el **número de tramos**, no el costo; sirve de línea base.

Función de costo multiobjetivo (`scalar_cost`): combinación lineal ponderada de minutos, costo (miles de COP) y CO_2 (centenas de gramos), con pesos `w = (tiempo, costo, emisiones)` ajustables por el usuario. El **costo monetario** se modela como **tarifa plana por abordaje** (un pasaje por tramo del mismo modo/ruta), no por km: un viaje Niquía La Estrella en metro cuesta un único $3.820.

4. Diseño del agente y del RAG

Agente (`core/agent.py`): loop agéntico manual con *tool calling* sobre un endpoint compatible con OpenAI (Groq / NVIDIA NIM / Ollama), con número máximo de pasos acotado (`T_MAX = 6`) y reintentos ante error 429. Expone **cuatro herramientas** (supera el mínimo de 2 que pide la rúbrica):

1. `plan_route` — ruta óptima A* entre dos estaciones (tramos, modo, paradas, minutos, costo, CO_2).
2. `get_metro_status` — si el Metro opera a una hora y tipo de día, y su horario.
3. `check_pico_y_placa` — si un vehículo tiene restricción según dígito, día y hora.
4. `search_docs` — recuperación sobre el corpus para reglas y normativa, con cita de la fuente.

RAG (`core/rag.py`): recuperador TF-IDF sobre `corpus/*.md` (metro, pico y placa, emisiones), con troceo por párrafos y similitud coseno. Es un recuperador ligero, suficiente para el tamaño del corpus; mejora sugerida: embeddings + índice vectorial (FAISS/Chroma). El *system prompt* obliga a usar las herramientas, a citar la fuente como `[archivo]` y a no inventar normas.

5. Resultados

Parte 1 — A* vs Dijkstra vs BFS (`python eval/benchmark_search.py`)

Evaluación sobre **2.520 casos** (todos los pares origen-destino × 3 perfiles de pesos × hora pico y valle):

Algoritmo	Nodos expandidos (media)
A*	**10.09**
Dijkstra	11.50
BFS	11.50

- A* encuentra el **mismo óptimo** que Dijkstra en el **100 %** de los casos (heurística admisible/consistente) **expandiendo menos nodos** (~12 % menos): la heurística aporta sin perder optimalidad.
- BFS tiene un **sobrecosto medio del 44,4 %** frente al óptimo, porque minimiza tramos y no costo ponderado.

Agente — evaluación (`eval/resultados.md` , `python eval/run_eval.py`)

Batería de **14 preguntas** (rutas, estado del Metro, pico y placa, normativa, una fuera de alcance y una no cubierta por el corpus):

- **Herramienta correcta: 14/14 (100 %).**
- **Citó la fuente: 7/7** de las preguntas que lo requerían.
- Casos de control: la pregunta fuera de dominio (mejor restaurante) **no** llama herramientas; la pregunta sobre la multa, que el corpus no detalla como dato central, se maneja con `search_docs` reconociendo lo que no está en los documentos.

6. Limitaciones

1. **Grafo simplificado:** nodos = estaciones Línea A; no se modelan Líneas B, cables, tranvía ni la red vial fina (hay soporte OSM opcional vía `scripts/download_osm.py`).
2. **Emisiones y velocidades son supuestos** (no inventario oficial); afectan el eje de CO_2 y deben calibrarse con AMVA/SIATA.
3. **Rutas de bus mapeadas a estaciones:** son corredores reales pero sus paradas se aproximan a las estaciones que sirven, no a GTFS exacto.
4. **RAG TF-IDF:** sin comprensión semántica profunda; sensible al vocabulario.
5. **Dependencia del LLM:** la pestaña del agente requiere `LLM_API_KEY` ; sujeta a límites de tokens del plan (se mitiga con reintentos y recorte de contexto). Las pestañas de rutas y benchmark funcionan sin clave.
6. **Normativa con vigencia temporal:** pico y placa y tarifas cambian por semestre/año; el corpus fija una fecha de consulta (13-nov-2026) y el agente solo afirma lo que está documentado.

7. Reproducibilidad

Código, datos y documentación en el repositorio. Ejecución de punta a punta descrita en `README.md` (entorno virtual, `requirements.txt`, `.env`, `streamlit run app.py`). Métricas reproducibles con `python eval/benchmark_search.py` (Parte 1) y `python eval/run_eval.py` (agente, requiere `LLM_API_KEY`).